Using Transfer Learning to Create Custom Object and Scene Detection: Measuring the Accuracy and Performance of Multiclass Convolutional Neural Networks

Alex Gallagher[1] Mark Quimba[1] Paul Abili[1] Devon Kaelberer[1] Khushika Shah[1]

[1] Dept. of Computer Science, UMBC

Abstract

This document is an analytical report that details the methodology and a series of experiments run against a custom implementation of an advanced pre-trained convolutional neural network. We consider prior work and research done in the field, including different applications of multiclass neural networks and similar projects completed in the past, such as those on the COCO2017 dataset. Utilizing transfer learning techniques on the architecture of a YOLO11N pre-trained model, we created a multiclass model that aims to recognize both a greater number of labels than the original model, as well as scene identification of a given image. Then, we detail our method of building the model, and the series of experiments we designed around it to validate its performance on training images. Finally, we consider our results and contextualize them in a broader range of limitations and setbacks. Our GitHub repository can be found at https://github.com/mclquimba/CMSC475Project and our interactive frontend can be found at https://nn-475-obje\ct-detection-model.streamlit.app/.

Keywords—neural network, convolutional neural network, multiclass neural network, multiclass classification, transfer learning, object detection, scene classification, COCO2017 dataset, Objects365 dataset, Places365 dataset

1. Introduction

As neural networks expand their capabilities across a wide range of interdisciplinary applications, a central concern of their increasing prominence is their accuracy with respect to the predictive classification of the features they have been assigned to identify. Across training epochs of a convolutional model with a massive dataset, we hypothesize that model performance will improve at learning and recognizing more features, although it will struggle to classify any with strong confidence due to the massive number of possible labels it needs to distinguish. To validate this hypothesis, we propose to build on top of an existing multiclass convolutional model and test its performative accuracy as the number of training iterations increases. As the model is expected to recognize more features per epoch, we anticipate that the model will perform better with those with many examples in the dataset and ones that match between pre-trained and trained datasets, but will struggle to recognize ones with few examples and dataset mismatch.

1.1. Outline of Project Goals

Our project goal has been to investigate, research, and build a functional multiclass convolutional neural network model. Utilizing a pre-trained model infrastructure called YOLO11N (You Only Look Once, Version 11 Nano), we aimed to investigate how well the model performs after training, as well as its scalability and performative accuracy across a greater range of images and labels it was originally trained on. While YOLO11N is pre-trained on the COCO2017 dataset, we used transfer learning to modify additional layers of our neural network to be trained on the Objects365 dataset and the Places365 dataset, for additional object and scene detection capabilities, respectively. On top of the original model's capabilities, our model is trained to recognize more features (such as those present in Objects365 that were not present in COCO2017) and be able to detect an image's scene, so that it can make predictions as to what environment the image takes place within.

1.2. Procedure and Reporting

To validate our results, we have run 15 epochs of our model against the Objects365 dataset and 20 epochs against the Places365 dataset. These datasets have millions of training examples between them, meaning that our model has effectively seen examples of many of the features it is expected to recognize several million times. Across these epochs, we have built graphs and visualized results with a wide variety of reporting techniques, aimed to show model improvement over time. By analyzing the relevant factors of this training data, as well as by considering drawbacks and

imperfections in our training datasets, we have been able to construct a comprehensive report of our findings, particularly contextualized within the scope and limitations of our design.

Additionally, we have developed an interactive frontend to visualize model performance against any series of custom images we choose to run it against. Open to anyone, this frontend allows us to visualize results in a clear and interactive manner, and in a way that validates some of our findings qualitatively. By utilizing this frontend against cherrypicked test images designed to validate concurrent accuracy, the model's ability to recognize a variety of items in bulk, and the model's recognition of rarer labels with fewer examples, the frontend has allowed us to present results in a refined and user-friendly way.

2. Related Work

To effectively contextualize our project within the broader field of deep learning, we will investigate the existing landscape of multiclass neural networks our project falls within. We begin by defining some core challenges of multiclass neural networks and some pre-existing strategies and approaches others have used to remediate them. Then, we analyze the implementation of these networks across diverse domains, from network security to medical diagnostics. Finally, we investigate the role of large-scale datasets, such as COCO and Objects365, in validating model scalability and performance.

2.1. Architectural Foundations of Multiclass Neural Networks

The evolution of multiclass classification is rooted in a larger transition from binary structures and functionalities to modular architectures rooted in branchable subproblems and multitasking. Anand et al. established that modular neural networks could provide more efficient classification for complex multiclass problems by decomposing large problems into smaller subtasks, increasing performance through modularity [1]. Building upon this, subsequent methodologies sought to refine performative accuracy through specialized algorithms and architectural techniques. Ou and Murphey investigated the fundamental patterns of multiclass classification, showing that neural networks must be fundamentally fluid and adaptable to maintain high performance as label counts increase [2]. Further innovations introduce hybrid models and architectures, such as Borisov and Korshunova's fuzzy neural network, which utilizes fuzzy logic to handle ambiguity across overlapping multiclass categories [3]. Even more, Bhardwaj et al. showed that genetic optimization techniques are effective ways to fine-tune multiclass classifiers to create superior results [4]. All of these help contribute to the research foundation of the architecture we have built our model upon and the strategies and techniques investigated in the past that have influenced its inherent design.

2.2. Interdisciplinary Applications and Case Studies

In general, the robustness of multiclass neural networks is best seen across their use in various high-stakes, data-rich environments. In the realm of cybersecurity, Baig et al. implemented a multiclass cascade of neural networks specifically aimed at intrusion detection, categorizing multiple concurrent attack vectors precisely and effectively [5]. In another high-stakes field, the medical industry has found great use of these technologies. Melin et al. developed a model that uses a complex vectorized algorithm for the multiclass classification of arrhythmias to create a reliable tool that can be used to analyze and identify patterns in cardiac systems [6]. In dermatology, Navarrete-Dechent et al. concluded that, among identifying skin conditions, improvement can still be achieved of multiclass convolutional neural networks at identifying rare labels [7]. Even recently, Saqi et al. utilized a U-net based multiclass model to evaluate non-small cell lung cancer, showing that these systems with proper training can have improved performance even in complex microenvironments [8]. Across all of these case studies, they all showcase the inherent ability for multiclass models to perform well given a strong foundational architecture and adequate training.

2.3. Large-Scale Relevant Datasets

Across modern architectures and models, such as YOLO, their performance is heavily dependent on the datasets used to benchmark them. For the YOLO model (the one utilized in this project), the COCO (Common Objects in Context) dataset has served as a standard for object detection and segmentation. Puri detailed the complexities of COCO segmentation, noting that models trained on it need to understand the difference between foreground and background items in a multiclass setting [9]. Beyond standard object detection, Veit et al. introduced COCO-Text, a different benchmark specifically aimed at text detection in natural scene images, highlighting the model's need for precision in its multiclass classification [10]. However, as label counts skyrocket, issues such as dataset imbalance become prominent. Soleimani and Mirshahzadeh investigated the classification of imbalanced data using deep neural networks, providing strategies to maintain accuracy even when certain classes have far fewer training examples than others [11]. By recognizing fallbacks in certain architectures we are reliant on, we have been able to navigate the challenges of our design by incorporating strategies that have been researched and integrated by other researchers in the past.

2.4. Motivation for Our Current Implementation

While the aforementioned studies demonstrate the power and potential of multiclass neural networks, there remains a notable gap in the performance analysis of real-time smaller architectures like YOLO11N when scaled to massive datasets like Objects365 and Places365. Our work seeks to address this by applying transfer learning to expand a model's label library while observing trade-offs like confidence and precision. By

integrating both object detection and scene identification, we aim to move beyond isolated classification and toward a more contextual understanding of visual environments.

3. Methods

To investigate the performance boundaries and scalability of multi-task convolutional architectures, we have developed a two-stage sequential transfer learning pipeline. As previously discussed, we chose to utilize YOLO11N as our foundational pre-trained model. On top of this infrastructure, we first executed deep transfer learning to establish a model that already has an increased ability to detect multiple objects across a wider range of hundreds of categories. Then, to create the scene classification model, we extracted the weight configuration from our best model trained from our object detection stage to learn different representations of scenery, utilizing even more transfer learning techniques along with additional training. These methods holistically construct the set of models we have built, both in multiclass classification using a convolutional neural network and scene detection with a similar foundational framework.

3.1. Object Detection

Our initial goal focused on expanding the core feature-extraction capabilities of the neural network to accommodate a massive vocabulary of diverse target classes. While our original ambition was to train our model for a duration of 100 epochs, due to hardware limitations from UMBC's cluster chip as well as time constraints, we were only able to train for 15 epochs.

To manage our optimization, we selected the AdamW (Adam with Decoupled Weight Decay) optimizer. This choice was driven by its superior capability in handling complex gradient environments without causing premature saturation. The optimization configuration was bound to a learning rate of 0.001, paired with a regularizing weight decay coefficient of 0.0005. This configuration performed the best out of the ones we tested because it was most able to balance generalization to new images with overfitting on the training set.

For our training dataset, we selected the Objects365 dataset, which contains 365 distinct, fine-grained categorical classes distributed across roughly 2 million high-resolution natural images. Given the immense scale of this corpus, we used the Ultralytics API for training the model. Incorporating this framework allowed us to streamline the underlying data pipeline to more efficiently manage our goals across the semester.

3.2. Scene Classification

Our scene classification architecture represents a custom, hybrid convolutional classification model that builds off the object detection model directly. Rather than training a scene classifier from completely randomly initialized weights, we implemented a layer-grafting technique to transfer learning from our other model. We systematically preserved the first 9 layers of the trained object detection backbone, which was already trained features from the Objects365 dataset across 15 epochs (as stated previously). We then stripped the downstream bounding-box regression layers and replaced them with a C2PSA (CSP Bottleneck with Parallel Multi-Head Self-Attention) module sourced from the Ultralytics framework. This module uses convolutional layers with self-attention mechanisms to allow the model to understand global features of the image overall, for an image-wide understanding of scenery.

To construct the functionality of our scene classification, we created a custom classification head that uses a series of convolutional layers, batch normalization layers, average pooling, dropout, a fully-connected linear layer, and a SiLU (Sigmoid Linear Unit) activation function for nonlinearity. This structural design creates a model that contains a blend of deep learning, downsampling, and regularization techniques to actualize a strong framework of scene classification on top of an existing object detection model.

Like with our modified object detection model, we used AdamW as our optimizer for the scene classification model. Additionally, we re-used the same hyperparameters of a weight decay of 0.0005 and a learning rate of 0.001. We utilized categorical cross-entropy loss as our objective (loss) function to penalize misclassifications of the environment labels. This model was trained for 20 total epochs against the Places365 dataset (which has around 1.8 million training images), ensuring deep coverage of global environments in our model.

4. Experiments

Now, we investigate the results and analysis we have been able to perform based on the model and research framework we have completed. First, we entail our analysis of the datasets we decided to build our models on, as well as our analysis of distributions and mismatches present within it. Then, we investigate some figures related to the training performance of our models. Further, we look at qualitative results we have been able to produce from our frontend, as well as some notable drawbacks of our model's performance with some examples.

4.1. Dataset Selection and Parsing

We selected three datasets to evaluate object detection performance across varying levels in scenes and annotations. YOLO11N is pre-trained on the baseline COCO2017 dataset which has approximately 330,000 images. Objects365, on the other hand, has approximately 2 million images and was selected because of its high annotation density and image quality. Finally, Places365 is included to incorporate the scene classification model and evaluate how YOLO would behave across diverse real world environments for general classification without bounding boxes. Table 1 summarizes the key properties of each dataset.

Table 1: Dataset Overview — COCO 2017 vs. Objects365 vs. Places365

Metric	COCO 2017	Objects365	Places365
Used for	Pre-trained	Transfer-Learning	Scene Classification
Total Images	330k	2M	1.8M
Total Categories	80	365	365 (scenes)
Annotation Type	Bounding Box, Segmentation Mask, Caption	Bounding Box	Scene Label
Image Resolution	(~640×480)	Variable (High-resolution)	256×256
Crowd Annotations	Yes (filtered)	No	N/A

Table 1. Dataset Overview — COCO2017 vs. Objects365 vs. Places365. Differences between these datasets across notable categories that our model has had to adjust for.

For COCO2017 and Objects365, annotations are loaded from their respective JSON files and parsed using a custom Python pipeline. As we approached the datasets, we excluded the annotations marked as fake or reflected to maintain a high annotation quality. For Places365, scene indices were loaded from their validation file and then mapped to scene categories.

4.2. Data Analysis

To study how detection will scale with scene density, we distributed all images into four object count bins based on the number of annotated objects per image. The bin boundaries are defined as:

Bin 1 – exactly 1 object

Bin 2 – 2 to 5 objects

Bin 3 – 6 to 10 objects

Bin 4 – 11 or more objects

Since overlap is a significant challenge for detection, we use these boundaries to oversee how the model will perform under different scene and object complexities including dense multi-object scenes and highly crowded scenes [12].

The three datasets do not have a perfect overlap as the datasets vary in distribution sizes and have class imbalance. COCO2017 has a relatively balanced distribution with most of the images falling into Bin 2 (2 to 5 objects). Images in this bin are mostly everyday photography images. More challengingly, Bin 4 consists of about 22% of the total images, with an average of 18 objects per image. Objects365 is skewed towards the denser bins (ones that fall into Bin 4), averaging around 19 objects per image in that bin. A very small number of images in the dataset fall into Bin 1 (exactly 1 object). Due to the density skew, this dataset is a harder benchmark than COCO for object detection.

Places365 is evaluated via an inference of the YOLO11N model and it has a different profile than the previous two datasets. The overall average detection rate was about 2 objects per image. The majority of images fell in Bin 1 and Bin 2, showing that there is a limited overlap between COCO's object categories and Places365's scene classification.

4.3. Class Distribution Analysis

Analyzing the top annotated classes per bin across the three datasets, we find that the person class is the dominant category in every multi-object bin (Bins 2–4). This class accounts for most instances in each image across the three datasets. This suggests that the most challenging bin across all datasets is a crowd detection problem rather than an object detection one. Models that struggle in Bin 4 are challenged by overlapping human figures, largely from occlusion between pedestrians and other dense object configurations.

Analyzing the bottom annotated classes, we find sparsity for rare categories: toaster and hair dryer in COCO2017, table tennis and curling in Objects365, and fork and snowboard in Places365. When the model is tested with these classes, the confidence threshold and the model's performance are low. While our model performs well at recognizing labels with a variety of adequate examples, it struggles to recognize ones with the fewest examples.

4.4. Model Performance

Now that we have analyzed the intricacies of the model's training datasets, we will analyze its performance through a series of metrics, graphs, and tables that relay its performance across epochs.

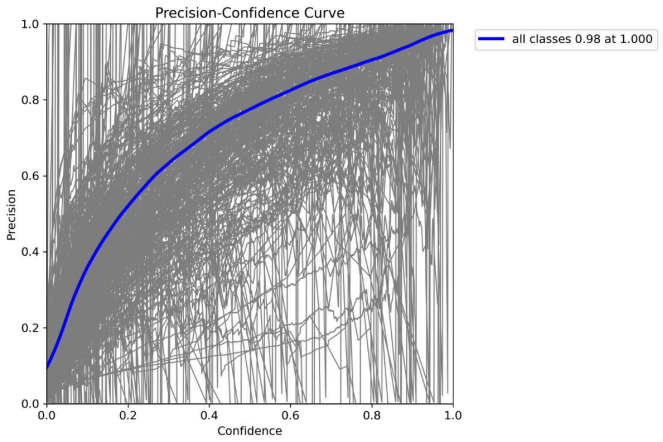

Figure 1. Precision–Confidence Curve. Performance of the object detection model after 15 epochs with varying confidence levels. As the model gets more confident in its classification, it performs more precisely (is more likely to be correct).

Figure 1 shows the trade-off between precision and confidence in the model's performance. There is a positive correlation between the model's precision as its confidence is varied. Therefore, generally, when the model is more

confident in a given label, it is more likely to be precisely correct. While some classes present do not have such a strong correlation, the average class (in blue) does recognize strong overall performance.

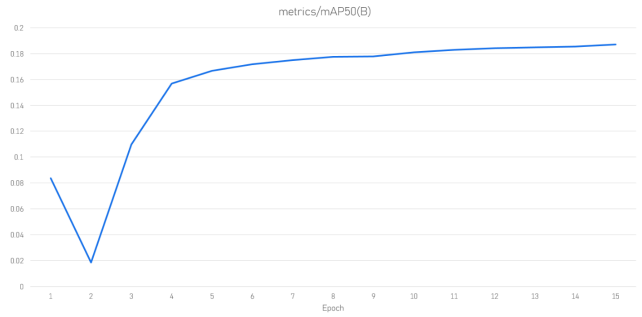

Figure 2. mAP50 by Epoch. mAP50 stands for mean Average Precision at an IoU (Intersection over Union) threshold of 50%. IoU threshold is a standard way of measuring the positioning of bounding boxes within a certain degree of accuracy. The object detection model's increasing performance indicates that it has improved with more accurate bounding boxes across epochs.

Figure 2 shows the mean average precision of the object detection model's performance across epochs by using 50% as a threshold (mAP50). This measures the percentage of overlap between the model's prediction and the ground truth label. If the bounding box overlap is at least 50%, it is evaluated to be correct by this metric. After the first epoch, our mAP50 decreases; however, after that the epoch steadily increases until it reaches 0.185 by epoch 15.

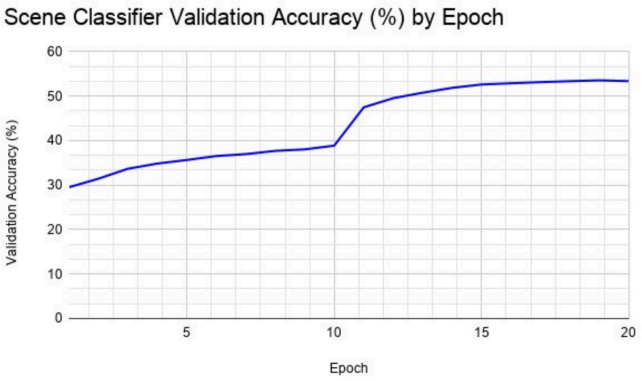

Figure 3. Scene Classifier Validation Accuracy by Epoch. Across training epochs, our model substantially improved performance at classifying scenes from Places365.

Figure 3 measures the validation accuracy for the scene classifier across epochs. In general, with more training epochs, the model continually performed better. This validates that, at least across the domain recognized in its training images, the model has improved accuracy over epochs. Alike our original hypothesis that model performance would be improved upon across training epochs but only to a limited extent, this curve demonstrates its validation. While the ultimate accuracy wavers around 53%, it shows that the model still cannot recognize so many different scene labels, and only can improve until where it guesses it correctly around half the time.

4.5. Output Evaluation

To evaluate our models further, we qualitatively tested their ability to classify various features across various test images. We hypothesized that when presented with images containing features with low training examples, our object detection model would either make a classification error or fail to classify the object altogether. We also realized that when an image had an ambiguous background, it would cause the scene detection model to misclassify it as something else. While additional training would have potentially helped the model's performance, we recognize that domain gap was the more likely cause of test misclassification. While our model's training iterations were performed on high-quality images with very specific features, the images we were testing it against did not follow the same formatting or structure, likely causing disparity. Nonetheless, we do recognize that more training would improve the model's accuracy of particularly rare classes with fewer training examples, that its parameters could not recognize without more training.

Figure 4 contains an example of our model classifying an image with a person. The person was easily classified with a high confidence level, but the tie was classified with a relatively low confidence level. The scene was completely misclassified as a chemistry lab, but this was likely more due to the ambiguous background.

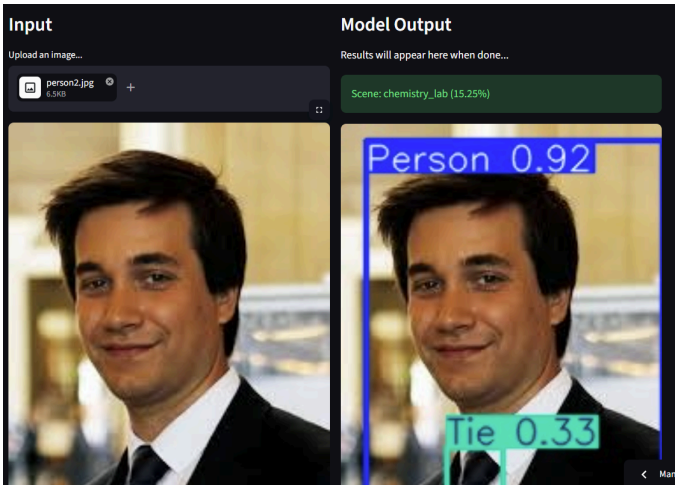

Figure 4. An image of a person wearing a suit and tie in an unknown location. The person and the tie are both successfully classified, with a confidence level of 0.92 and 0.33 respectively. The scene is incorrectly classified as a chemistry lab.

Figure 5 presents an example of where our model is able to classify multiple items in an image. While people and the other items classified in this image were in the pre-trained COCO2017 dataset, balloons were only introduced with our

fine-tuning through Objects365. This test image demonstrates our model's adapted learning to recognize more features with high confidence than what was originally in its pre-trained series of possible labels.

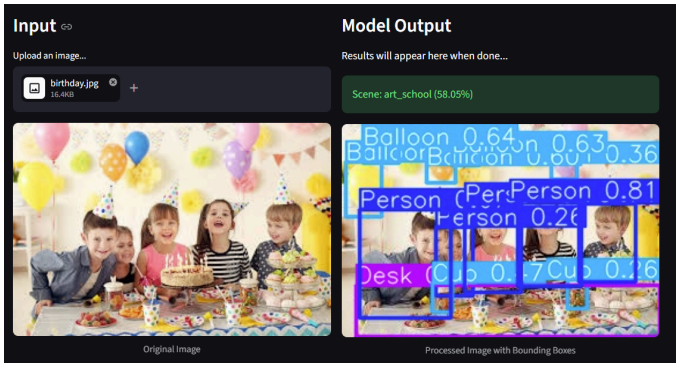

Figure 5. An image of 5 children. Within the image there are multiple balloons, a cake, cupcakes and party hats. The children and various items are classified within the image. The confidence level is relatively high for most objects. The scene is determined to be an art school, which is incorrect.

Note: Regrettably, improving overlapping bounding box labels was outside of the scope of the project, as it was hard-coded in the pre-trained architectural infrastructure we built our model from.

Figure 6 presents an example of our model attempting to classify an object with one of the lowest training examples. Our model is completely unable to classify the toaster as an object at all. Instead, it assumes the toaster is a part of the background and is a bank vault. The white background is another factor which causes difficulty for our model to classify objects from various features, likely causing this misclassification. As aforementioned, our model has only been trained on natural images, so being able to recognize one with a completely white background causes it to misclassify.

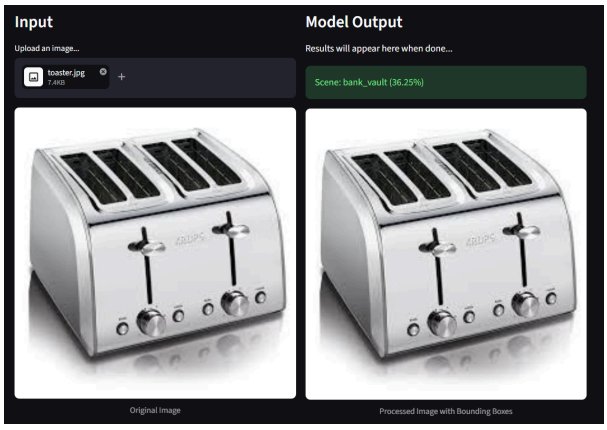

Figure 6. An image of a toaster with a white background. The toaster is completely unclassified. The scene is classified as a bank vault which is incorrect.

5. Conclusion

In this paper, we have constructed a pre-trained multiclass object detection neural network to demonstrate as a proof-of-concept that transfer learning can be used to create a new model that can classify both objects and scenes. We began with the pre-trained YOLO11N and created two models serving different purposes utilizing its architectural and weight configuration. The first model we created expanded the domain of features that the model was expected to recognize, using Objects365. The second used transfer learning from our object detection model to create a scene classifier that uses the entire image to understand scenes, trained on Places365. Experimentally, we determined that much of our model's weakness, particularly those relating to certain classes and misclassification, were due to underlying biases and mismatching from the datasets we utilized. Then, we demonstrated that our models have been able to improve performance over training epochs by validating our results quantitatively. Then, we qualitatively supplemented those results by constructing an interactive, user-friendly frontend and recording its results on cherry-picked test images. These results demonstrated our model's ability to adapt and recognize images outside of its original pre-trained dataset, as well as struggle to recognize ones far outside its trained domain, and ones with few training examples. While we were unable to train our model for a substantial amount of epochs, we still were able to assert that the model meaningfully improved its performance across new images.

6. Limitations

While our model is validated as an effective proof-of-concept for both transfer learning and multiclass convolutional classification, we have encountered drawbacks that limit the effectiveness of our results. From our data collection and pre-training process as well as the transfer learning and evaluation stages, we have realized that our model does possess some weaknesses that invalidate its performance on some images. Knowing these concerns, we have ultimately tried to offset the effect these limitations would have on our results by designing our model against them, although they still have affected some aspects of our model overall.

6.1. Choice of Pre-Trained Model and Datasets

First, our dataset selection and some inherent differences between them limited our model's performance with some degree of clashing overlap. Our data collection process has always been limited by the datasets that were available to be trained on that fit our goals. The requirements inherent to open-source multiclass object detection models and storage limitations on the class chip meant that COCO2017 and by extension YOLO11N were the optimal choice given our limitations as undergraduate researchers. Because YOLO11N was pre-trained on COCO2017 (and based on its uneven distribution of label examples, as discussed previously), it had

a large bias towards classes that appeared more frequently in its pre-trained examples. This, therefore, meant our model was cursed to struggle with classes with very few examples, forced to ignore their misclassification when training due to it affecting so little of its overall accuracy. Limited to the datasets upon which it was trained on, our model is trained to recognize classes that possess more training examples than others, biasing its performance towards the majority.

Furthermore, the choice of the Nano version of the YOLO11 model was made out of necessity, given both timing restrictions during the semester and storage restrictions on the UMBC Chip computing cluster. This had the expected tradeoff of limiting the amount of parameters that the model could learn. The result of using the Nano version is that our model was able to be trained faster at the cost of lower absolute performance after many training iterations.

6.2. Training Errors

Another limitation of this study was the amount of training epochs that we were able to run on the scene and object classification datasets. The Places365 and Objects365 are both very large datasets which have around 2 million images each. This resulted in a single training epoch containing around 2 million photos which took an incredibly long time to complete. Throughout our training process, we experienced errors on the class chip that would crash the training loop after a session of up to only several hours. Given our need to perform adequate training on the model and our continuous need to keep retraining the model from scratch, we had to pivot some parts of learning to Google Colab to finish results by our deadline. These training setbacks meant that the group was only able to run only 15 and 20 epochs for the object detection and scene classification models, respectively. While the model did demonstrate learning from its small sample of training epochs, we initially wanted to run a training loop of up to 100 epochs for each classifier to get an idea of the maximum possible performance of each classifier. Unfortunately, however, we were unable to do that.

6.3. Testing Errors

One of the main issues with our model's performance was its invariability to perform on images only like its training set. As seen in our qualitative results on the frontend, our model struggles to recognize classes with few labels and ones with differing backgrounds and image styles than it used to in training. Looking at the datasets we utilized, while they do differ in some ways, such as by label type, count, and distribution, they do not differ in ways of only showing natural images, and having some classes be under-represented with examples. Therefore, we recognize that transfer learning is in no way similar to domain generalization. While the model is able to pick up different classes and labels, and can even perform different functionalities than was originally intended, it cannot properly perceive changes outside of its scope of reference. If given a completely blank background (something which is not present in any of its trained datasets), the model always performs poorly to recognize both objects and classes within. Similarly, if in testing the model is suddenly expected to recognize all of its rarest labels, it cannot learn afterwards to classify them, as it does not adapt to the interactivity of its frontend. Despite these errors with our testing, we do perceive that our model performs very well in the domains it has been adapted to recognize through the process of transfer learning.

References

[1] Anand, R., Mehrotra, K., Mohan, C. K., & Ranka, S. (1995). *Efficient classification for multiclass problems using modular neural networks*. IEEE Transactions on Neural Networks, 6(1), 117–124. https://ieeexplore.ieee.org/document/363444

[2] Ou, G., & Murphey, Y. L. (2007). *Multi-class pattern classification using neural networks*. Pattern Recognition, 40(1), 4–18. https://doi.org/10.1016/j.patcog.2006.04.041

[3] Borisov, V. V., & Korshunova, K. P. (2019). *Multiclass classification based on the convolutional fuzzy neural networks*. Communications in Computer and Information Science, 1093, 226–233. https://doi.org/10.1007/978-3-030-30763-9_19

[4] Bhardwaj, A., Tiwari, A., Bhardwaj, H., & Bhardwaj, A. (2016). *A genetically optimized neural network model for multi-class classification*. Expert Systems with Applications, 60, 211–221. https://doi.org/10.1016/j.eswa.2016.04.036

[5] Baig, M. M., Awais, M. M., & El-Alfy, E. M. (2017). *A multiclass cascade of artificial neural network for network intrusion detection*. Journal of Intelligent & Fuzzy Systems, 32(4). https://doi.org/10.3233/JIFS-169230

[6] Melin, P., Amezcua, J., Valdez, F., & Castillo, O. (2014). *A new neural network model based on the LVQ algorithm for multi-class classification of arrhythmias*. Information Sciences, 279, 483–497. https://doi.org/10.1016/j.ins.2014.04.003

[7] Navarrete-Dechent, C., Liopyris, K., & Marchetti, M. A. (2021). *Multiclass artificial intelligence in dermatology: Progress but still room for improvement*. Journal of Investigative Dermatology, 141(5), 1325–1328. https://doi.org/10.1016/j.jid.2020.06.040

[8] Saqi, A., Liu, Y., Politis, M. G., Salvatore, M., & Jambawalikar, S. (2024). *Combined expert-in-the-loop--random forest multiclass segmentation U-net based artificial intelligence model: Evaluation of non-small cell lung cancer in fibrotic and non-fibrotic microenvironments*. Journal of Translational Medicine, 22(1). http://dx.doi.org/10.1186/s12967-024-05394-2

[9] Puri, D. (2019). *COCO dataset stuff segmentation challenge*. 2019 5th International Conference on Computing, Communication, Control and Automation (ICCUBEA). https://ieeexplore.ieee.org/abstract/document/9129255

[10] Veit, A., Matera, T., Neumann, L., Matas, J., & Belongie, S. (2016). *COCO-Text: Dataset and benchmark for text detection and recognition in natural images*. Cornell University. https://doi.org/10.48550/arXiv.1601.07140

[11] Soleimani, M., & Mirshahzadeh, A. S. (2023). *Multi-class classification of imbalanced intelligent data using deep neural network*. EAI Endorsed Transactions on AI and Robotics, 2(1). https://doi.org/10.4108/airo.3486

[12] Landeen, T., & Gunther, J. (2024). Deep learning based image overlap detection. *IEEE*. https://fileserver-az.core.ac.uk/download/220134542.pdf